The Applied AI Engineer

The Applied AI Engineer

From Probability to Production Agents

Sridhar Mukkandi

Draft

The Applied AI Engineer: From Probability to Production Agents

Copyright © 2026 Sridhar Mukkandi. All rights reserved.

No part of this book may be reproduced, stored or transmitted in any form without written permission from the author, except for brief quotations in reviews and articles.

Independently published.

About the examples in this book. Kaveri Kart, its customers and its staff are imagined. Every number about Kaveri Kart, and every price marked as an example, is assumed, to explain the ideas clearly; none is a measurement of a real company or product. Facts about real companies, products and incidents are cited.

Products and services change quickly. Model names, versions and prices were checked on the date given in the version register at the back of the book. All trademarks belong to their respective owners.

The information in this book is provided without warranty. The author assumes no responsibility for errors or omissions, or for damages resulting from its use. Nothing in this book is legal, security-compliance or financial advice.

Text set in Source Serif 4, Inter and JetBrains Mono, all under the SIL Open Font License.

Contact: sridhar.authorhub@gmail.com

Table of contents

Preface vi

How to use this book viii

I The job and the foundations xi

1 What an applied AI engineer does 1

2 Probability and statistics you will actually use 12

3 Classical machine learning: small, fast and cheap 22

4 Deep learning in one chapter 32

5 From words to tokens and vectors 40

Appendices 49

Answers 49

References 54

About the author 55

Preface

Many people want to become AI engineers, and most of them start the same way. They watch a video, copy a chatbot tutorial, connect it to a vector database, and get a demo working over a weekend. Then they try to put it in front of real users, and it falls apart. The bot invents a refund policy. It costs ten times more than expected. Nobody can say whether last week's change made it better or worse. A customer pastes a strange message, and the bot does something nobody planned.

This book is about the work that starts after the demo. It is a complete path, from the basic probability you need, through how large language models work, to designing AI agents that are safe to run in a real business. It is written for engineers, not for researchers. Every idea is here because you will use it at work or be asked about it in an interview.

Who this book is for

- **You are starting out.** You know some Python, and you want to become an AI engineer. Read the book in order. Part I gives you every foundation the rest of the book needs, and nothing more.
- **You are a software engineer.** You build backends, apps or data pipelines, and now your team wants AI features. Skim Part I and spend most of your time in Parts III to VIII.
- **You work in data science or machine learning.** You know models, but not agents, retrieval or the new protocols. Start at Part II or Part III.

What makes this book different

Most AI books teach tools. Tools change every few months, so those books go out of date quickly. This book teaches the judgement behind the tools, and keeps the details that change often in a dated register at the back.

An AI system is a chain of small decisions. Which team should get this email? Is this the right document? Is it safe to issue this refund, or should a person check first? Each decision needs three things: a number you can trust, a line drawn by what mistakes cost, and a safe way out when the system is unsure. You will meet these three things in Part I, and they come back in every part after it, from model choice to agent design.

You will also build one system from start to finish. **Kaveri Kart** is an imagined online store that sells home and kitchen products across India. Its customers write in English, Hindi and Hinglish. Through the book you will build its customer-support agent: it answers questions from the store's policy documents, looks up orders, and issues refunds behind a human check. By the last chapter, you will have a complete project you can explain in an interview, with every number in it justified.

How the book is organised

1. **The job and the foundations.** What an AI engineer does, and the probability, machine learning, deep learning and language basics underneath it.
2. **How LLMs work.** Transformers, training, reasoning models, long context, serving, and how to choose a model.
3. **Talking to models.** Prompts and context, structured outputs, tool calling, and your first tests.
4. **Knowledge and retrieval.** Embeddings, vector databases, and RAG that works.
5. **Agents.** From fixed workflows to agents, memory, and many agents working together.
6. **Protocols and frameworks.** MCP, A2A, and how to choose a framework.
7. **Trust.** Security, guardrails, private data, and testing agents properly.
8. **Production.** Turning a prototype into a service, running it, cost and speed, fine-tuning, and small models.
9. **Case studies.** Real systems that worked and real systems that failed, told as design reviews.
10. **Career.** Your portfolio, the job market, and interview preparation.

What you need

A laptop, basic Python, and an account with one model provider. Running every example in this book costs very little, and the cost of each one is shown so that nothing surprises you. You do not need a GPU.

Sridhar Mukkandi

How to use this book

Each chapter follows the same shape, so you always know where to find things.

The boxes

In this chapter you will learn. Three points at the start of each chapter, so you know what to look for.

Key idea. One lesson worth remembering, with a thick bar on its left. The bold headline is the lesson; the sentence under it explains why it is true. At the end of the chapter, a **Keep these** box lists the chapter's key ideas again.

Project step: Kaveri Kart. What we add to the running project in this chapter. If you are building along, do this step before moving on.

Decide. A real choice an AI engineer has to make, with the reasoning for each option.

What breaks in production. The common ways the chapter's idea goes wrong once real users arrive. These boxes are short, and they are some of the most useful pages in the book.

Numbers you must defend. Every AI system is full of numbers someone chose: how many test cases, which threshold, how long a text chunk. In an interview, or in a design review, you will be asked why. This box lists the numbers from the chapter and how to justify each one.

Exercises and interview questions. Try them before you turn the page. Answers to both are at the back of the book.

Code

Code in this book is short and is written in Python. Each listing runs as printed, once you install the package it imports and supply your own data where the text says so. You do not need to type the code to understand the chapter: the text explains what each part does.

Numbers and prices

Every number about Kaveri Kart is an assumed example, chosen to make the idea easy to see. Prices marked as example prices are made up, to keep the arithmetic simple. Real prices change often, so check your provider’s current price page before you plan a budget.

PART I

The job and the foundations

These five chapters give you the ground the rest of the book stands on. You will see what an applied AI engineer does all day, and make your first call to a model. Then you will learn the parts of probability, machine learning, deep learning and language processing that you will actually use at work. None of it is research-level. If an idea is here, a later chapter or a real job needs it.

CHAPTER 1

What an applied AI engineer does

Kaveri Kart is a small online store in Mysuru. It sells kitchen and home products across India: pressure cookers, steel tiffin boxes, bedsheets, mixer grinders. On a normal day, its six-person support team answers about 800 emails and chat messages. During the Diwali sale, that number jumps to 4,000 a day, and the team falls days behind.

The owner, Meena, decides to "add AI". A developer friend connects a chatbot to the website over a weekend. It is impressive for about a week. Then the problems start:

- It tells a customer that a used pressure cooker can be returned within 30 days. The real policy says 7 days, and only for unused items.
- It answers a customer who wrote in Hindi in English, and a customer who wrote in Hinglish in formal Hindi.
- Nobody can say how many of its answers are right. The team only hears about the wrong ones when customers complain.
- At the end of the month, the bill from the model provider is four times what anyone guessed.

None of this happened because the model was weak. The model wrote fluent, polite replies. Everything around the model was missing: the store's real policies, a way to test answers, a rule for when to hand a case to a person, and a budget. Building those things is the job of an **applied AI engineer**, and it is what this book teaches.

IN THIS CHAPTER YOU WILL LEARN

- What an applied AI engineer does, and how the role differs from a data scientist or an ML engineer.
- The three questions behind every AI system you will build.
- How to make your first call to a large language model, and what it costs.

The model is only one part of the system

Think about a busy restaurant. The cook matters, but a good cook alone does not make a good restaurant. Someone has to plan the menu and write the recipes. Someone has to keep the store room stocked with the right ingredients. Someone tastes the food before it leaves the kitchen, checks hygiene, and keeps an eye on what each plate costs. If any of these is missing, even a great cook sends out bad food.

An AI system works the same way. The **model** is the cook. A **large language model**, or **LLM**, is a program trained on a huge amount of text, which can read a message and write a reply. Around the model, an AI engineer builds everything else (Figure 1.1):

- **Instructions and context.** What the model is told about its job, and which facts it is given for this particular question. These are the recipes and the ingredients.
- **Knowledge.** The store's policies, product details and order records, found and handed to the model when needed. This is the store room.
- **Tools.** Ways for the model to look up an order or start a refund. These are the ovens and knives.
- **Tests.** Checks that tell you whether the answers are right, before customers see them. This is tasting the food.
- **Guardrails.** Rules that stop the system from doing harmful things, such as leaking a customer's address. This is hygiene.
- **Cost and speed.** What each answer costs, and how long the customer waits. This is the bill.

The weekend chatbot at Kaveri Kart had only the cook. That is why it failed, and it is why companies hire AI engineers instead of just buying access to a model.

● KEY IDEA

The model is one part of an AI system, and usually not the part that fails.

Most failures come from what is missing around the model: the right facts, tests, limits on what it can do, and a budget.

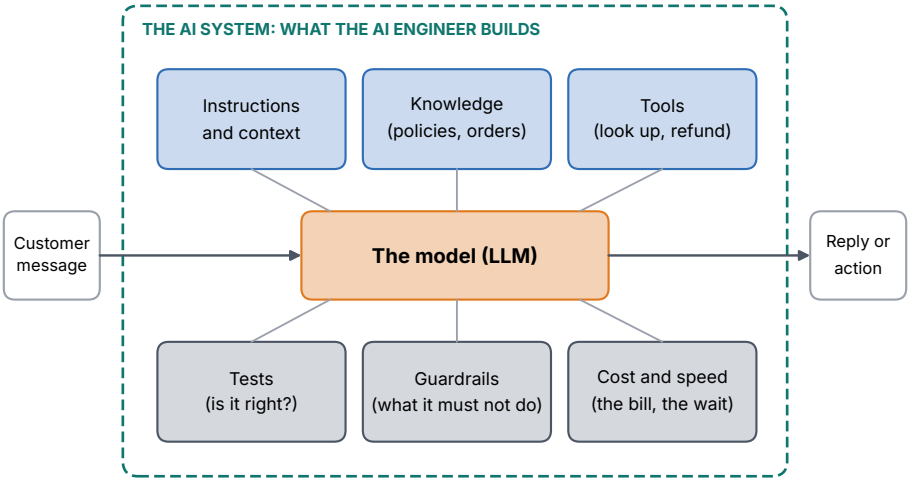


Figure 1.1. An AI system is much more than the model. The model sits in the middle; the AI engineer builds everything around it.

Three kinds of AI jobs

Job titles in AI are confusing, because companies use them differently. It helps to sort the work into three kinds, by where each person spends most of their time.

1. **Researchers** invent new kinds of models and new ways to train them. They work at a small number of labs and universities, and most have a PhD.
2. **Machine learning engineers** train models on a company's own data and run them in production. They spend their time on data pipelines, training jobs and model servers.
3. **Applied AI engineers** build products on top of models that already exist. They choose a model, give it the right context and tools, test it, make it safe, and ship it.

The writer Shawn Wang described the line between the last two roles simply: the ML engineer's work mostly stops once the model is serving answers, and the AI engineer's work starts there (Wang 2023). In India, the same role often appears as "GenAI engineer" at IT services companies, and as "AI engineer" or "applied AI engineer" at product companies and startups.

The lines blur in practice. An applied AI engineer sometimes trains a small model (you will do this in Chapter 3), and an ML engineer sometimes writes prompts. But the centre of the applied AI engineer's job is

clear: **turn a capable model into a system that works reliably for real users.**

What the work looks like in a week

Here is a realistic week for the engineer who takes over Kaveri Kart's support system.

1. **Monday: read real conversations.** Read 50 recent chats from start to end. Note every wrong or awkward answer, and group them: wrong policy, wrong language, made-up order status, too long.
2. **Tuesday: fix the biggest group.** Most wrong policy answers come from the model not having the policy at all. Add a step that finds the right policy section and gives it to the model with the question.
3. **Wednesday: test the fix.** Run the change against a fixed set of 200 past questions with known good answers. The share of correct answers moves from 71% to 88%, and nothing that was right before is now wrong.
4. **Thursday: decide what the system may do alone.** Questions about delivery dates can be answered automatically. Refunds above ₹2,000 will be prepared by the system but approved by a person.
5. **Friday: check cost and speed.** Look at the cost per conversation and the slowest replies. Find that very long chats are expensive, and plan a change for next week.

Notice how little of this week is about the model itself. Most of it is reading, testing, deciding and measuring. That is normal. One hiring guide for AI engineers calls "no evals", meaning no habit of testing AI output systematically, the strongest warning sign in a candidate (Nakrani 2026). This book spends a lot of time on testing for that reason.

Three questions behind every AI system

Every system you build in this book, from a simple email sorter to an agent that issues refunds, has to answer three questions. Keep them in mind from the first chapter, because each part of the book adds to the answers.

1. Does it work? You cannot answer this by trying a few examples and feeling happy. You need a set of test cases with known good answers, and a way to score the system against them. If you change a prompt or a model, you run the tests again. Chapter 2 gives you the basic statistics for this, and Chapter 15 builds your first real test suite.

2. Should it act by itself? Some actions are cheap and easy to undo, like answering a question about delivery times. Others are costly or permanent, like sending money or deleting data. For each action, the system can do one of three things:

- **act**: do it alone;
- **review**: prepare it, and let a person check it;
- **escalate**: hand it to a person straight away.

Where to draw the lines between these three depends on how sure the system is and on what each mistake costs. Chapter 3 shows how to draw them with simple arithmetic.

3. What happens when it is wrong? Every AI system makes mistakes. The question is whether a mistake is small and caught quickly, or large and silent. A good design limits the damage: the support bot cannot see payment details, refunds have a maximum amount, and every action is logged so a person can check it later.

> KEY IDEA
>
> **Every AI system must answer three questions: does it work, should it act alone, and what happens when it is wrong?**
>
> A demo answers none of them. A production system answers all three, with numbers.

Writing or deciding?

When people think of AI, they think of writing: essays, emails, code. But look again at Kaveri Kart's support work. For each incoming message, the system must:

- decide which language it is in;
- decide what kind of request it is: refund, delivery, product question or complaint;
- decide whether the customer is angry enough to need a senior person;
- decide which policy applies;
- decide whether a refund is allowed;
- and finally, write a reply.

Five of these six steps are decisions with a small number of possible answers. Only the last one is writing. This pattern repeats in almost every real system, and it matters because decisions and writing need different

tools. A decision with a few fixed answers can often be made by a small, cheap, fast model, and its quality is easy to measure: it was either right or wrong. Writing needs a large model, and its quality is harder to judge.

A good AI engineer notices which parts of a system are decisions, and handles them carefully: with a clear set of answers, a measured accuracy, and a rule for when to ask a person. You will see this idea in every part of the book.

Your workbench

You need very little to start. Set up these five things once, and every chapter after this one will work.

1. **Python.** Install a recent version of Python 3 from python.org, or use the one that comes with your system.
2. **A code editor.** VS Code is free and popular, but any editor you like is fine.
3. **Git.** It keeps a history of every change, so you can undo a bad prompt change in seconds. Commit your work often.
4. **An account with a model provider.** Anthropic, OpenAI and Google all sell access to their models through an **API**, a way for your program to send a request over the internet and get an answer back. Create an account and an **API key**, a long secret string that identifies you and lets the provider bill you.
5. **A safe place for the key.** Never write the key inside your code, and never commit it to git. Anyone who finds it can spend your money. Store it in an **environment variable**, a named value your operating system keeps for your programs:

```
export ANTHROPIC_API_KEY="your-key-here"
```

On Windows, set it once with `setx ANTHROPIC_API_KEY "your-key-here"` and open a new terminal.

A word on notebooks. Jupyter notebooks are good for trying ideas, and you will use them. But a system that serves customers must run as normal scripts and services, with tests. Hiring managers often reject candidates whose work never left a notebook (Alian Software 2026). In this book, ideas start in a notebook and move into scripts as soon as they work.

Your first call to a model

Install the provider's Python package with `pip install anthropic`. Then save this as `first_call.py` and run it:

```
import anthropic

MODEL = "your-model-name"   # copy a current model name from the provider's docs

client = anthropic.Anthropic()   # reads the key from ANTHROPIC_API_KEY

reply = client.messages.create(
    model=MODEL,
    max_tokens=300,
    messages=[{"role": "user",
               "content": "Can I return a pressure cooker after 10 days?"}],
)
print(reply.content[0].text)
print(reply.usage.input_tokens, reply.usage.output_tokens)
```

Here is what each part does:

- `anthropic.Anthropic()` creates a **client**, the object that talks to the provider's servers. It finds your key in the environment variable by itself.
- `model` says which model to use. Model names change every few months, so this book never prints one in its code. Copy a current name from your provider's documentation.
- `max_tokens` sets the longest reply you are willing to pay for. A **token** is a small piece of text, usually a short word or part of a longer word. Chapter 5 explains tokens properly. For now, think of 100 tokens as about 75 English words.
- `messages` is the conversation so far. Here it has one message, from the user.
- The reply contains the model's text, and `usage`, which tells you how many tokens went in and how many came out.

OpenAI's and Google's Python packages look almost the same: a client, a model name, a list of messages, and a reply with text and token counts. Once you can use one, you can use the others.

Run the program and read the answer. It will probably be polite and sensible, something like "Most stores accept returns within 7 to 30 days, but please check the store's return policy." That sounds fine until you remember the question was about Kaveri Kart. The model has never seen Kaveri Kart's policy. It answered with a general guess. If you ask it to be more specific, it may invent a policy that sounds real. This is called a **hallucination**:

a confident answer that is not supported by any fact the model was given. Giving the model the right facts is the subject of Part IV.

Run the same program three times. The wording will change a little each time, because the model picks its words with some randomness. Chapter 2 explains why, and later chapters show how to control it.

What one answer costs

Providers charge by the token, with one price for tokens you send (**input**) and a higher price for tokens the model writes (**output**). Prices are usually given per million tokens.

Let us work out the cost with example prices. Suppose a model charges ₹250 per million input tokens and ₹1,250 per million output tokens. (These are example prices only; real prices vary a lot between models and change often.) Suppose one support reply uses 300 input tokens and 150 output tokens.

1. Input cost: 300 × ₹250 ÷ 10,00,000 = ₹0.075.
2. Output cost: 150 × ₹1,250 ÷ 10,00,000 = ₹0.1875.
3. Total: about ₹0.26, or 26 paise, per reply.

That sounds tiny. Now multiply. At 4,000 messages a day during the sale, it is about ₹1,050 a day, or about ₹31,500 a month. And real support replies use far more than 300 input tokens, because the model must also be given the policy text, the order details and the earlier messages in the chat. With 3,000 input tokens per reply, the same month costs about ₹1,12,500.

This is how the weekend chatbot's bill surprised everyone. Cost in AI systems grows with the number of tokens you send, and you send far more than you think. From this chapter on, every project step notes its cost.

● KEY IDEA

Work out the cost per task before you build, and check it after.

A few paise per answer becomes lakhs of rupees a month at real volumes, and most of the cost comes from the context you send, not the reply.

The project: Kaveri Kart's support agent

By the end of this book, Kaveri Kart's support system will look like Figure 1.2. Each part of the book adds a piece:

- In Part I, you collect and label real-looking customer messages, and build a small model that sorts them by type.
- In Parts II and III, you choose a model, write its instructions, and give it its first tool: looking up an order.
- In Part IV, the system learns to answer from Kaveri Kart's own policy documents, and to say "I don't know" when the documents do not cover a question.
- In Parts V and VI, it becomes an agent that can take actions, such as preparing a refund, behind a human check.
- In Parts VII and VIII, you protect it from attacks, test it properly, deploy it as a service, and bring its cost down.

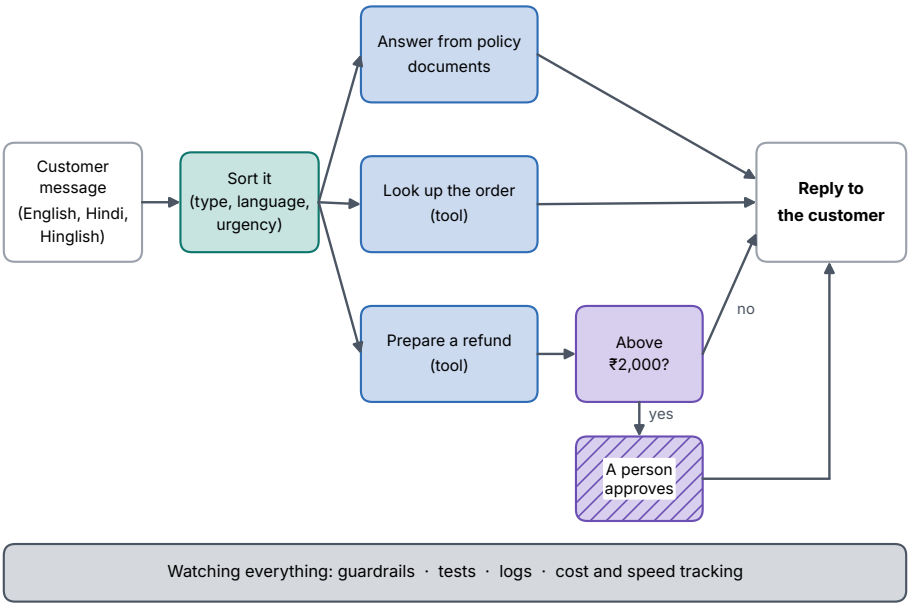


Figure 1.2. The Kaveri Kart support system at the end of the book. Messages are sorted first; questions are answered from policy documents; order lookups and refunds go through tools; refunds above a limit wait for a person. Tests, logs and cost tracking watch everything.

PROJECT STEP: KAVERI KART

1. Make your first call to a model, as shown above, and note the token counts.
2. Write down 20 questions a Kaveri Kart customer might ask. Mix English, Hindi and Hinglish, and include a few angry ones. For example: "My mixer stopped working after 2 weeks, I want a replacement", "Mera order 5 din se pending hai", "Do you deliver to Shillong?"
3. Send each question to the model with no extra information. Mark each answer as *correct*, *wrong* or *made up*. Since the model knows nothing about Kaveri Kart, expect many made-up answers. This is your starting point, and you will measure every later improvement against it.
4. Work out the cost of the 20 calls, using your provider's real prices.

WHAT BREAKS IN PRODUCTION

- **The model invents facts.** Without the business's real documents, the model fills gaps with believable guesses. Customers believe them.
- **Nobody measures quality.** Without a fixed test set, every change is judged by gut feeling, and quality drifts down without anyone noticing.
- **Costs grow quietly.** Long conversations and large documents push the number of input tokens up every month.
- **Private data leaks into prompts and logs.** Customer phone numbers and addresses end up in prompts, logs and test files that many people can read.

NUMBERS YOU MUST DEFEND

- **Which model, and why?** Name the model tier you chose (large and slow, or small and fast) and the reason: quality on your test questions, cost per answer, or speed.
- **Cost per answer.** Show the arithmetic: input tokens, output tokens, the price of each, and the total at expected daily volume.
- **What the system must never do alone.** List these actions, such as refunds above a limit or changes to an address, and say why each one needs a person.

KEEP THESE

- **The model is one part of an AI system, and usually not the part that fails.** Most failures come from what is missing around the model: the right facts, tests, limits on what it can do, and a budget.
- **Every AI system must answer three questions: does it work, should it act alone, and what happens when it is wrong?** A demo answers none of them. A production system answers all three, with numbers.
- **Work out the cost per task before you build, and check it after.** A few paise per answer becomes lakhs of rupees a month at real volumes, and most of the cost comes from the context you send, not the reply.

EXERCISES

1. List the six parts around the model shown in Figure 1.1. For a food-delivery app's support bot, give one example of each.
2. A model costs ₹400 per million input tokens and ₹1,600 per million output tokens (example prices). Each answer uses 2,000 input tokens and 200 output tokens. What does one answer cost? What do 10,000 answers a day cost for a 30-day month?
3. For a bank's customer-support assistant, name two actions it may take alone, two it should prepare for a person to review, and one it should escalate straight away.

INTERVIEW QUESTIONS

1. What is the difference between an ML engineer and an applied AI engineer?
2. Our chatbot gives confident answers that turn out to be wrong. What are the first three things you would check?
3. How would you estimate the monthly cost of an LLM feature before building it?

Next chapter: the probability and statistics you need to answer the first question, *does it work?*, with numbers instead of feelings.

CHAPTER 2

Probability and statistics you will actually use

Two engineers at Kaveri Kart each write a new set of instructions for the support model. They test both on the same 20 customer questions. Version A gets 17 answers right. Version B gets 15 right. A is better, so the team ships A.

A week later, they test both again on 20 different questions. This time B gets 17 right and A gets 15. Which version is really better?

Neither test can tell. With only 20 questions, a difference of two answers can easily happen by luck. A batsman can score three fifties in five innings and then three single-digit scores in the next five, without becoming a worse player. This chapter gives you the few ideas from probability and statistics that prevent this mistake. It also covers a handful of others that come up every week in AI work. There are no proofs here, and no formulas you cannot work out on paper.

IN THIS CHAPTER YOU WILL LEARN

- What a probability means, and why rare events make most alarms false.
- How sure you can be about a test score, using one simple method called the bootstrap.
- How a language model uses probability to pick its next word.

A probability is a share of many cases

The weather app says “70% chance of rain”. It does not rain. Was the app wrong?

It was not, because a probability does not describe one day. It describes many similar days. A **probability** is a number between 0 and 1 (or 0% and 100%) that says how often something happens across many cases like this one. “70% chance of rain” means: of all the days when the app says 70%, it should rain on about 70 out of 100.

The same is true for AI systems. Suppose a model says a customer message is “80% likely to be a refund request”. On one message, it will be either right or wrong. Only by looking at many messages where the model said 80% can you check whether it was right about 80 of every 100. You will use this idea in the last section of this chapter.

When something is rare, most alarms are false

Kaveri Kart has a problem with fake refund requests. Some people claim a product arrived broken when it did not. Suppose:

- 1 in every 200 refund requests is fake;
- a fraud checker catches 90% of fake requests;
- it also wrongly flags 4% of genuine requests.

The checker flags a request. What is the chance that it is really fake?

Most people guess something close to 90%. The right answer is about 10%. The easiest way to see why is to count, using 10,000 refund requests:

1. Fake requests: 1 in 200 of 10,000 is **50**.
2. Fake requests the checker catches: 90% of 50 is **45**.
3. Genuine requests: 10,000 − 50 is **9,950**.
4. Genuine requests flagged by mistake: 4% of 9,950 is **398**.
5. All flagged requests: 45 + 398 is **443**.
6. Fake requests among the flagged ones: 45 out of 443, which is about **10%**.

There are so many genuine requests that even a small mistake rate produces far more false alarms than real catches. How common something is before you look at any evidence is called its **base rate**. Here the base rate of fake requests is 1 in 200. Whenever someone tells you how good a checker is, ask for the base rate before you believe what its alarms mean.

● KEY IDEA

When the thing you are looking for is rare, most alarms are false.

A checker that catches 90% of fake refunds is right only about 1 time in 10 when it raises an alarm, because genuine requests are 199 times more common. Count it out with 10,000 cases to see this.

Accuracy can fool you

Accuracy is the share of all cases a model gets right. It sounds like the obvious way to measure a model, but with rare events it misleads you badly.

A "checker" that never flags anything is right on all 9,950 genuine requests and wrong on the 50 fake ones. Its accuracy is 9,950 ÷ 10,000 = 99.5%. It also catches zero fraud. So for rare events, use two better measures, both based on the counting above:

- **Precision**: of all the requests the checker flagged, what share were really fake? Here, 45 ÷ 443, about 10%. Low precision means the fraud team wastes its time on false alarms.
- **Recall**: of all the fake requests, what share did the checker flag? Here, 45 ÷ 50 = 90%. Low recall means fraud gets through.

Precision and recall usually pull against each other. Make the checker stricter and it flags fewer requests: precision goes up, but recall goes down. Which one matters more depends on what each mistake costs, and Chapter 3 shows how to use costs to choose. In interviews, you will often be asked for both numbers, never accuracy alone.

How sure is your test score?

Back to the two engineers. Version A got 17 out of 20 questions right, a score of 85%. How much would that score move if we had tested on 20 *different* questions?

A cricket example helps. A batsman's average after 5 innings tells you very little; after 100 innings, it tells you a lot. The more cases you see, the less luck matters. A test score works the same way. We need a way to turn "17 out of 20" into a range: "the real score is probably somewhere between this and that."

There is a simple method that works for almost any score you will meet, called the **bootstrap**. The idea is to pretend to repeat your test many times, using only the results you already have:

1. Write down your test results: 17 ones (right) and 3 zeros (wrong).
2. Draw 20 results from this list at random, putting each one back after you draw it. Some results will be drawn twice and some not at all. This gives you a slightly different "test".
3. Work out the score of that new test.
4. Repeat steps 2 and 3 ten thousand times.

5. Sort the ten thousand scores. Throw away the lowest 2.5% and the highest 2.5%. What remains is the range.

This range is called a **95% confidence interval**. In plain words, it is the range where the real score probably lies. In Python, the whole method is a few lines:

```
import numpy as np

def bootstrap_range(results, rounds=10_000):
    """results: one number per test case, 1 for right and 0 for wrong."""
    rng = np.random.default_rng(0)
    results = np.asarray(results)
    scores = [rng.choice(results, len(results)).mean() for _ in range(rounds)]
    return np.percentile(scores, [2.5, 97.5])

print(bootstrap_range([1] * 17 + [0] * 3))   # 17 right out of 20
```

The answer is about 70% to 100%. For version B, with 15 out of 20, it is about 55% to 90%. The two ranges overlap almost completely. With 20 questions, the test simply cannot tell the two versions apart.

How many test cases do you need? Figure 2.1 shows how the range shrinks as the test set grows, for a system that is right about 80% of the time:

- With 20 cases, the range is about ±17 percentage points around the score.
- With 50 cases, about ±11.
- With 200 cases, about ±5.5.
- With 1,000 cases, about ±2.5.

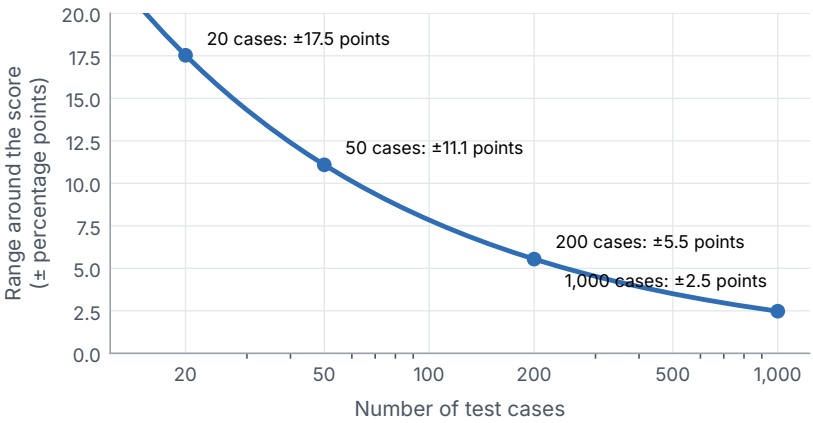


Figure 2.1. How wide the range around a test score is, for a system that is right about 80% of the time. The range shrinks quickly at first and then slowly: to halve it, you need four times as many test cases.

Notice the shape of the curve. To make the range half as wide, you need four times as many cases. That is why a test set of 200 to 500 well-chosen cases is a common, practical size for a first version of an AI system: big enough to see a difference of 5 to 10 points, small enough that a person can check the answers.

● KEY IDEA

A score on a small test set is mostly luck.

With 20 test cases, a score of 85% could really be anything from 70% to 100%. Always report the range with the score, and grow your test set until the range is narrow enough to make your decision.

Comparing two versions fairly

When you compare two versions of a system, there is a better way than working out two separate ranges. Test both versions on the **same** questions, and look at how they differ question by question. This is called a **paired comparison**.

Suppose the engineers now test A and B on the same 200 questions. On most questions, both versions give the same result: both right, or both wrong. Those questions tell you nothing about which version is better. What matters are the questions where they disagree:

- On 14 questions, A is right and B is wrong.
- On 6 questions, B is right and A is wrong.

So A is ahead by 8 questions out of 200, or 4 percentage points. Is that real? Use the same bootstrap. For each question, write +1 if only A was right, −1 if only B was right, and 0 if they agreed. Pass this list of 200 numbers to `bootstrap_range`. The answer is a range from about 0 to 8.5 points.

The range touches zero, so we cannot yet be sure A is better. The engineers test on 200 more questions, and the same pattern holds. With 400 questions, the range becomes about 1 to 7 points. Now it no longer includes zero, and they can say with reasonable confidence that A is better, by a few points.

Three habits keep comparisons honest:

1. **Use the same test set** for both versions. If the test questions change, you are measuring the questions, not the versions.
2. **Fix the test set before you start changing things.** If you keep adjusting the test set until your favourite version wins, the result means nothing.

3. **Look at the disagreements by hand.** Reading the 20 questions where A and B differ usually teaches you more than the score does.

How a language model uses probability

Probability is not only for testing. It is also how a language model writes.

Give a model the words "Your order has been". Before it writes the next word, the model works out a probability for every word it knows. For example:

- "shipped": 42%
- "delivered": 25%
- "cancelled": 12%
- "packed": 9%
- "delayed": 7%
- "refunded": 5%

(These numbers are made up for the example; a real model spreads its probability over tens of thousands of possible next pieces of text.)

Each probability depends on the words so far. A probability that depends on what you already know is called a **conditional probability**. A language model is, at its heart, a machine for one conditional probability: *the chance of each next word, given all the words before it.*

The model then picks one word. If it always picked the most likely word, it would write "shipped" every time, and long texts would become dull and repetitive. So by default it picks at random, in proportion to the probabilities: "shipped" about 42 times in 100, "delivered" about 25 times, and so on. This random choice is called **sampling**, and it is why the same question gives slightly different answers each time you ask.

You can control how adventurous the choice is with a setting called **temperature** (Figure 2.2):

- A **low temperature**, such as 0.5, makes the likely words even more likely. In our example, "shipped" rises from 42% to about 66%. The answers become more predictable.
- A **temperature of 1** uses the model's probabilities as they are.
- A **high temperature**, such as 1.5, flattens the probabilities. "Shipped" falls to about 33%, and rarer words get picked more often. The answers become more varied, and more likely to go wrong.

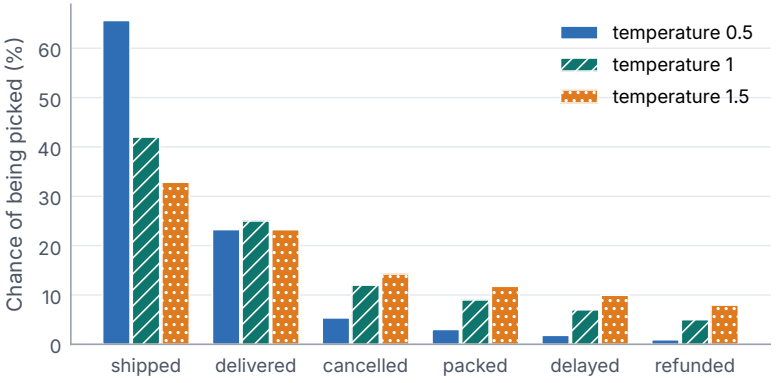


Figure 2.2. The same next-word probabilities at three temperatures. A low temperature sharpens the choice towards the most likely word; a high temperature spreads it out.

For support replies, data extraction and decisions, you want predictable answers, so use a low temperature. For brainstorming product descriptions, a higher temperature gives more variety. Even at temperature 0, which should always pick the most likely word, a model served by a provider can give slightly different answers to the same question, for technical reasons you will meet in Chapter 11. Never build a test that expects an exact word-for-word answer.

When the model says 80%, is it right 80% of the time?

Many AI systems give a probability with each answer: "refund request, 80%". A system whose probabilities match reality is called **calibrated**. When a calibrated model says a message is 80% likely to be a refund request, about 80 of every 100 such messages really are refund requests.

Nothing forces a model to be calibrated, so you must check. The check uses the idea from the start of this chapter, that a probability is a share of many cases:

1. Take a few hundred past messages where you know the right answer.
2. Put them in groups by the chance the model gave for "refund request": 0–10%, 10–20%, and so on, up to 90–100%.
3. In each group, count how many messages really were refund requests.
4. Draw the result: the chance the model gave along the bottom, how often it really happened up the side (Figure 2.3).

Points on the diagonal mean the probabilities can be trusted. Points that are flatter than the diagonal mean the model is **overconfident**: its numbers are more extreme than reality, so it sounds more sure than it should be. Large models are often overconfident, and the "confidence" an LLM

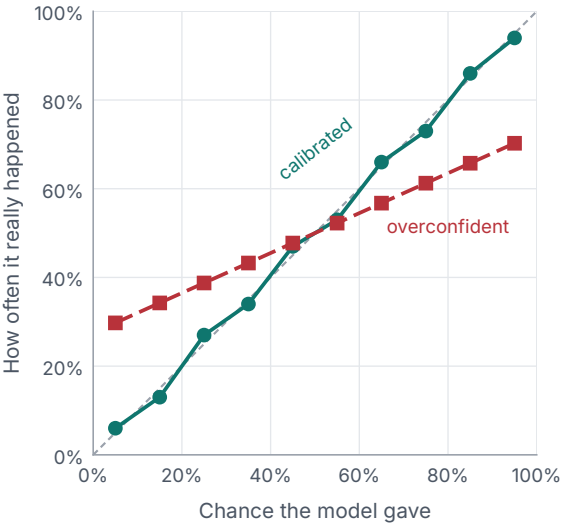


Figure 2.3. Checking calibration. Each point is a group of messages. For a calibrated model, the points sit on the diagonal: when it says 80%, it happens about 80% of the time. The overconfident model's numbers are too extreme: when it says 95%, it happens only about 70% of the time, and when it says 5%, it still happens about 30% of the time. (Illustrative data.)

writes in its answer ("I am 95% sure") is usually not calibrated at all. You will check LLM confidence properly in Part VII.

Why does this matter so much? Because in the next chapter, you will make the system act on its probabilities: route the message automatically above one value, send it to a person below it. If the model's 80% really means 60%, every one of those rules is wrong.

● KEY IDEA

Before you act on a model's probability, check that it means what it says.

Group past cases by the probability the model gave, and count how often it was right in each group. If it says 80% but is right only 60% of the time, every rule built on its numbers will be wrong.

PROJECT STEP: KAVERI KART

1. Create a test set of 200 Kaveri Kart customer messages. Write them yourself, or ask a model to write varied examples and then edit them by hand. Include English, Hindi and Hinglish messages, short and long ones, and a few angry or confusing ones.
2. Label each message by hand with one of five types: *refund*, *delivery*, *product question*, *complaint*, *other*. This labelled set is your **ground truth**, the answers you trust, and the next chapter trains a model on data like it.
3. Count the share of each type. These are your base rates. If complaints are only 5% of messages, remember the lesson about rare events.
4. Save `bootstrap_range` in a file called `stats.py`. You will use it in almost every chapter.

WHAT BREAKS IN PRODUCTION

- **Tiny test sets.** Teams compare versions on 10 or 20 examples and ship the "winner". The difference is usually luck.
- **Test cases leak into the prompt.** If an example from the test set is also used as an example in the model's instructions, the model has seen the answer, and the score is too high.
- **The test set stops matching real traffic.** Customers start asking about a new sale or a new product, and the old test set no longer covers what they ask.
- **An average hides a weak group.** The system scores 90% overall but only 60% on Hindi messages. Always look at the score for each important group separately.

NUMBERS YOU MUST DEFEND

- **Test-set size.** "We used 200 cases, which gives a range of about ±5.5 points at our score. That is enough to see the 10-point differences we care about."
- **The range with every score.** "Version A scored 86%, with a 95% range of 81% to 90%."
- **Base rates.** "Complaints are 6% of our messages, so we report precision and recall for complaints separately, as well as overall accuracy."
- **Temperature.** "We use a low temperature for support replies, because we want the same question to get the same answer."

KEEP THESE

- **When the thing you are looking for is rare, most alarms are false.** A checker that catches 90% of fake refunds is right only about 1 time in 10 when it raises an alarm, because genuine requests are 199 times more common. Count it out with 10,000 cases to see this.
- **A score on a small test set is mostly luck.** With 20 test cases, a score of 85% could really be anything from 70% to 100%. Always report the range with the score, and grow your test set until the range is narrow enough to make your decision.
- **Before you act on a model's probability, check that it means what it says.** Group past cases by the probability the model gave, and count how often it was right in each group. If it says 80% but is right only 60% of the time, every rule built on its numbers will be wrong.

EXERCISES

1. A spam filter catches 98% of spam and wrongly flags 1% of normal emails. Only 1 in 100 emails is spam. Of the emails it flags, what share are actually normal? Count it out with 10,000 emails.
2. Your system gets 45 out of 50 test questions right. Without running any code, is the real score more likely to be within ±2 points of 90% or within ±10 points? Why?
3. Two versions are tested on the same 300 questions. Version A alone is right on 9 questions, and version B alone is right on 7. What would you tell your manager?

INTERVIEW QUESTIONS

1. Our fraud model has 99% accuracy. Is it good?
2. How many test cases do you need to compare two prompts?
3. What does temperature do, and what value would you use for a customer-support bot?
4. What does it mean for a model to be calibrated, and how do you check it?

Next chapter: small, fast models that learn from examples, and how to turn their probabilities into actions using what each mistake costs.

CHAPTER 3

Classical machine learning: small, fast and cheap

Every message that reaches Kaveri Kart's support team first has to go to the right person. Refund requests go to the refunds desk, delivery questions to the logistics desk, product questions to the product team, and complaints to a senior agent. During the sale, one person spends the whole day just reading and sorting messages.

You could send each message to a large language model and ask, "Which of these five types is this?" It would work well. But at 4,000 messages a day it costs money, every answer takes a second or two, and the model's own sense of how sure it is cannot be trusted (Chapter 2).

There is an older, simpler tool for this job. Train a small model on a few thousand messages that people have already sorted. It can then sort a new message in a few milliseconds, at almost no cost, and give a probability you can check. This chapter shows how such models learn, how to test them without fooling yourself, and how to turn their probabilities into actions.

IN THIS CHAPTER YOU WILL LEARN

- How a model learns from labelled examples, and how to test it honestly.
- Two classical models every AI engineer should know: logistic regression and gradient-boosted trees.
- How to decide, using costs, when the model should act alone and when it should ask a person.

Learning from examples

Think of a kirana shopkeeper who gives credit to regular customers. Nobody gave the shopkeeper a written rule for who pays back on time. Over the years, they noticed patterns: customers who buy regularly, who have a steady job nearby, who paid back last time. Now, when a new customer asks for credit, the shopkeeper makes a good guess in seconds.

Machine learning works the same way. Instead of writing rules by hand, you give a program many examples where the answer is already known, and it finds the patterns itself. Two words come up constantly:

- A **label** is the known answer for an example. For Kaveri Kart, the label of a message is its type: *refund*, *delivery*, *product question*, *complaint* or *other*.
- A **feature** is a piece of information the model can use to guess the label. For a message, the features are usually its words. For a refund request, features could also be numbers such as the order value and the days since delivery.

Learning from labelled examples is called **supervised learning**. A model that picks one label from a fixed list is called a **classifier**. Almost every decision in an AI system, such as which team, which language, or whether something is safe, can be made by a classifier.

Turning text into features

A classifier works with numbers, so a message must first become a list of numbers. The simplest way is to count words. Make a list of every word that appears in your messages. For each message, write down how often each word appears. A message about a broken cooker gets a high count for "broken" and "cooker", and zero for "delivery".

Plain counting gives too much weight to common words like "the" and "my", which appear in every message. A small improvement called **TF-IDF** fixes this: it gives each word a weight that is high when the word is frequent in *this* message but rare across *all* messages. "Refund" gets a high weight in a refund request; "the" gets almost none.

For Indian customer messages, one more trick helps a lot. Hinglish has no fixed spelling: "nahi", "nahin" and "nai" all mean the same thing. If you count short groups of letters instead of whole words ("nah", "ahi", and so on), these spellings share most of their pieces, and the model learns that they are similar. These letter groups are called **character n-grams**.

Practice papers, mock tests and the final exam

To know whether a model has really learned, you must test it on examples it has never seen. Think of how a student prepares for a board exam:

- They learn from **practice papers**, where they can see the answers.

- They take **mock tests** to check their progress and decide what to study next.
- The **final exam** has questions they have never seen, and it is taken only once.

Data for machine learning is split the same way (Figure 3.1):

1. **Training set** (about 70% of your labelled examples): the model learns from these.
2. **Validation set** (about 15%): you use it to compare settings and choose thresholds.
3. **Test set** (about 15%): you use it once, at the end, to report how good the model is.

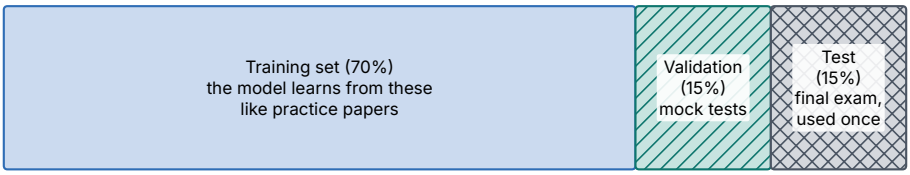


Figure 3.1. Labelled examples are split three ways. The model learns from the training set; you tune choices on the validation set; the test set is kept aside and used once, for the final score.

If you keep checking the test set while you tune, you slowly tune the model to the test set itself, the way a student who sees the exam paper in advance gets a high mark without learning anything.

That second problem has a name: **leakage**, which means information about the answer slipping into the model's training in a way that will not happen in real use. Leakage is the most common reason a model scores brilliantly in testing and poorly in production. Watch for three kinds:

- **Duplicates.** The same message, or almost the same message, appears in both the training set and the test set. Customers often send the same complaint twice. Remove near-duplicates before you split.
- **Future information.** A feature that is only known after the answer. If your data has a column "refund_issued", it will predict "refund request" perfectly, but a new message does not have that column yet.
- **Shared customers.** If one customer's messages are in both sets, the model may learn that customer's writing style instead of the message type. Split by customer, not by message.

KEY IDEA

Test a model only on examples it has never seen, and use the test set once.

If information about the answers leaks into training, through duplicates, future columns or shared customers, the test score will look excellent and mean nothing.

Logistic regression: points for each clue

The most useful classical model is **logistic regression**. Despite the long name, the idea is simple, and it works like a credit score:

1. Each feature gets some points, learned from the training examples. For the *refund* label, "refund" might be worth +3 points, "money back" +2.5, and "where is" −1.
2. For a new message, add up the points of all its features.
3. Turn the total into a probability between 0% and 100%. A high total gives a probability near 100%, a very negative total gives one near 0%.

With five labels, the model keeps five sets of points, one per label, and turns the five totals into five probabilities that add up to 100%. The message "paisa wapas karo, cooker toot gaya" (give my money back, the cooker broke) might get *refund* 81%, *complaint* 12%, *product question* 4%, *delivery* 2%, *other* 1%.

How does the model learn the points? It starts with every point at zero, makes guesses on the training examples, measures how wrong they were, and adjusts the points a little to be less wrong. It repeats this many times. This guess-check-adjust loop is how almost every model in this book learns, including the large language models in Part II.

Logistic regression is fast, cheap, easy to understand (you can read its points and see why it decided something), and usually gives fairly well-calibrated probabilities. That makes it the right first model for almost any text classification task.

Gradient-boosted trees: many small questions

Some decisions depend on numbers more than words. To flag a suspicious refund request, Kaveri Kart might use the order value, the days since delivery, the number of refunds this customer has asked for, and whether the delivery photo was taken.

For this kind of data, in tables with columns of numbers and categories, the strongest common choice is a **gradient-boosted tree** model. A **decision tree** is a chain of yes-or-no questions: "Is the order value above ₹5,000? Has this customer asked for more than three refunds this year?" One tree is weak. Boosting builds hundreds of small trees, one after another, each trying to fix the mistakes of the trees before it. Together, they are very accurate. Popular libraries include XGBoost, LightGBM, and the `HistGradientBoostingClassifier` in scikit-learn.

A simple guide:

- For **text**, start with TF-IDF and logistic regression.
- For **tables of numbers and categories**, start with gradient-boosted trees.
- For both, measure first, and only try something more complex if the simple model is not good enough.

Overfitting: learning the answers instead of the lesson

A student who memorises last year's answer key scores full marks on last year's paper and fails this year's. A model can do the same. If it is flexible enough, it can learn the exact training examples, including their accidents, instead of the general pattern. This is called **overfitting**.

You can see overfitting in your numbers: the score on the training set is much higher than on the validation set. For example, 99% on training and 78% on validation. The fixes are the usual ones: more training examples, a simpler model, or settings that stop the model from leaning too heavily on any single feature. The validation set is what tells you whether a fix worked.

A baseline in a few lines

Here is a complete message classifier in Python, using the scikit-learn library (`pip install scikit-learn`). `texts` is your list of messages and `labels` is the list of their types.

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline

X_train, X_test, y_train, y_test = train_test_split(
    texts, labels, test_size=0.2, stratify=labels, random_state=0)

model = make_pipeline(
    TfidfVectorizer(analyzer="char_wb", ngram_range=(2, 4)),   # letter groups
    LogisticRegression(max_iter=1000))
model.fit(X_train, y_train)

probs = model.predict_proba(X_test)   # one probability per type, per message
```

`stratify=labels` keeps the share of each type the same in both sets, which matters when some types, like complaints, are rare. A model this simple is called a **baseline**: the score any fancier approach must beat to be worth its cost. Always build one first. Sometimes the baseline is good enough, and you have saved weeks of work.

Measuring a classifier

A single accuracy number hides too much. For each label, look at precision and recall (Chapter 2). Then look at which labels get confused with which. A table of these mix-ups is called a **confusion matrix**, but you can read the important part in plain words. For example, in Kaveri Kart's test set:

- Of 40 delivery messages, 34 were sorted as *delivery*, 5 as *refund* and 1 as *complaint*.
- Of 12 complaints, only 8 were sorted as *complaint*; 4 went to *refund*.

The second line is the worrying one. An angry customer whose complaint goes to the refunds desk waits longer and gets angrier. Recall on complaints matters more than recall on anything else. This is why you always look at the mistakes label by label, and weigh them by what they cost.

From probability to action: when to ask a person

The classifier gives probabilities. The business needs actions. For each message, the system can:

- **act**: send it straight to the team the model chose;
- **review**: send it to a person who sorts it by hand;
- **escalate**: send it straight to a senior person, because it cannot wait.

Where should the line between *act* and *review* be? It depends on two costs. In our example, suppose:

- A message sent to the wrong team costs about **₹30**: someone reads it, sends it on, and the customer waits longer.
- A person sorting one message by hand costs about **₹6** of their time.

Now count, using 100 messages that the model sorts with a probability of 85%:

1. If the system acts alone, about 15 of the 100 go to the wrong team. Cost: 15 × ₹30 = ₹450.
2. If a person sorts all 100 instead, the cost is 100 × ₹6 = ₹600.

At 85%, acting alone is cheaper. Try 70%: acting alone sends 30 to the wrong team, costing ₹900, which is more than ₹600. At 80%, the two are equal: 20 × ₹30 = ₹600. So **80% is the line** (Figure 3.2). Above it, let the system act. Below it, ask a person.

The counting always works out to the same simple rule, which you can use for any action:

> **Act alone when (chance of being wrong) × (cost of a wrong action) is less than (cost of asking a person).**

For our numbers: at 80%, the chance of being wrong is 20%, and 20% × ₹30 = ₹6, exactly the cost of asking a person.

Two things make this rule work in practice. First, the probabilities must be calibrated, as you checked in Chapter 2. If the model says 85% but is really right only 65% of the time, the arithmetic above is wrong and too many messages go astray. Second, the costs are estimates. Try each cost at half and at double its value, and see how far the line moves. If it hardly moves, the exact costs do not matter much. If it moves a lot, spend time getting the costs right.

The *escalate* door uses a separate check. Any message the classifier marks as a complaint with a probability above 30%, or that mentions words like "consumer court" or "legal notice", goes straight to a senior agent. Missing

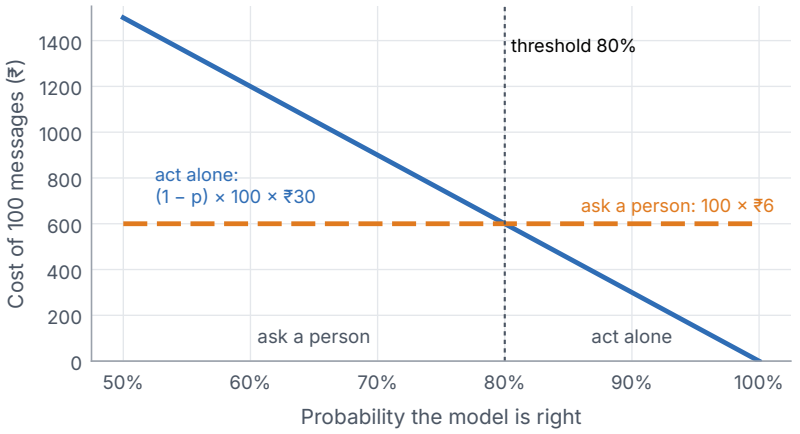


Figure 3.2. The cost of handling 100 messages, for each probability the model gives. Acting alone is cheap when the model is sure and expensive when it is not; asking a person costs the same every time. The lines cross at 80%: that is where the threshold goes.

an angry customer costs so much more than a senior agent's minute that even a modest probability is enough.

● KEY IDEA

Let the system act alone only when its expected mistake costs less than asking a person.

If a wrong routing costs ₹30 and a person's check costs ₹6, act alone only above 80% probability. The line comes from costs, not from habit.

⊣○ DECIDE

A small classifier or a large language model for sorting messages?
Choose a **classifier** when you have a fixed list of labels, a few thousand labelled examples (or can create them), high volume, and a need for speed, low cost and checkable probabilities.
Choose an **LLM** when the labels change often, you have very few examples, or the decision needs reasoning over long text or outside knowledge.
A common middle path: use an LLM to label a few thousand past messages, have a person check a sample of its labels, and train a classifier on the result. You pay for the LLM once, not on every message.

PROJECT STEP: KAVERI KART

1. Using your 200 labelled messages from Chapter 2, plus more if you can write or generate them (aim for 1,000 or more), train the baseline classifier above.
2. Report precision and recall for each of the five types on the test set, with the bootstrap range for each.
3. Check calibration on the validation set, as in Chapter 2.
4. Estimate your own costs for a wrong routing and for a manual check. Work out the *act* threshold. What share of messages would the system route alone?
5. Compare costs. At example prices from Chapter 1, sorting each message with an LLM might cost about 13 paise, or about ₹15,600 a month at 4,000 messages a day. The classifier runs on a small server for a fixed monthly cost. Write down both numbers.

WHAT BREAKS IN PRODUCTION

- **The world changes and the model does not.** A new sale, a new product line or a new kind of complaint appears, and the classifier keeps sorting it into old boxes. Watch the share of messages in each type, and retrain when it shifts.
- **Labels disagree.** Two support agents label the same message differently. If people cannot agree, the model cannot learn. Write a short labelling guide, and check how often two people agree.
- **The rare, important label is ignored.** With complaints at 5% of messages, a model can score high accuracy while missing most complaints. Track recall on the important label on its own.
- **Leakage.** The test score is excellent because of duplicate messages or a column that gives away the answer.

NUMBERS YOU MUST DEFEND

- **The act threshold.** "A wrong routing costs about ₹30 and a manual check about ₹6, so we route alone above 80%. Halving or doubling the costs moves the line between 70% and 90%."
- **Recall on complaints.** "Complaints are 5% of messages but the costliest to miss, so we track complaint recall separately. It is 86%, range 75% to 94%."
- **Training-set size.** "We used 2,000 labelled messages. Adding the last 500 changed validation accuracy by less than one point, so more labels would not help much now."

KEEP THESE

- **Test a model only on examples it has never seen, and use the test set once.** If information about the answers leaks into training, through duplicates, future columns or shared customers, the test score will look excellent and mean nothing.
- **Let the system act alone only when its expected mistake costs less than asking a person.** If a wrong routing costs ₹30 and a person's check costs ₹6, act alone only above 80% probability. The line comes from costs, not from habit.

EXERCISES

1. A dataset of refund requests has a column "agent_approved" filled in after a person reviewed each request. A model using this column predicts fraud almost perfectly. What has gone wrong?
2. A wrong automatic refund costs ₹500 on average. A person's check costs ₹25. Above what probability of the refund being valid should the system approve alone?
3. Your classifier scores 97% on the training set and 81% on the validation set. What is happening, and what would you try?

INTERVIEW QUESTIONS

1. When would you use a classical ML model instead of an LLM?
2. What is data leakage? Give an example from a real project.
3. How do you choose the probability threshold at which a model acts automatically?
4. Your classifier has high accuracy but customers say complaints are being missed. What do you look at?

Next chapter: what happens when the input is too messy for words and points, such as photos and raw text, and how neural networks learn from it.

CHAPTER 4

Deep learning in one chapter

Some Kaveri Kart customers do not write at all. They send a photo of a dented pressure cooker, or a voice note in Kannada saying the mixer makes a strange noise. The classifier from Chapter 3 cannot use either. It needs features someone has defined, like word counts or the order value, and nobody can write a useful list of features for the pixels of a photo.

Deep learning solves this. A deep learning model takes raw input, such as the pixels of a photo, the sound of a voice or the letters of a sentence, and learns its own features from examples. Every large language model is a deep learning model, so this chapter gives you the core ideas in plain words. You will not build networks from scratch in this book. You need to understand them well enough to use them, choose them, run them, and explain their limits.

IN THIS CHAPTER YOU WILL LEARN

- How a neural network is built from small units, and how it learns.
- Why neural networks are needed for photos, sound and free text, and when they are not needed.
- What a model needs in GPU memory, and how to estimate it on paper.

A team of small checkers

Recall logistic regression from Chapter 3: give each feature some points, add them up, and turn the total into a probability. That single unit, a weighted sum followed by a squashing step, is called an **artificial neuron**. One neuron can only learn simple patterns, where more of something always pushes the answer the same way.

Many real patterns are not like that. Think about a payment for ₹1 on a new card. Large payments can be suspicious, but so can tiny ones, because fraudsters often test a stolen card with a ₹1 payment first. Normal payments sit in the middle. One neuron that says “bigger is riskier” cannot learn “both very small and very large are risky”.

A **neural network** solves this with a team of neurons arranged in layers (Figure 4.1):

1. The first layer has many neurons, each checking something simple. One learns "is the amount very large?" Another learns "is the amount very small?"
2. The next layer combines their answers: "suspicious if either of those is true."
3. The last layer gives the final probabilities.

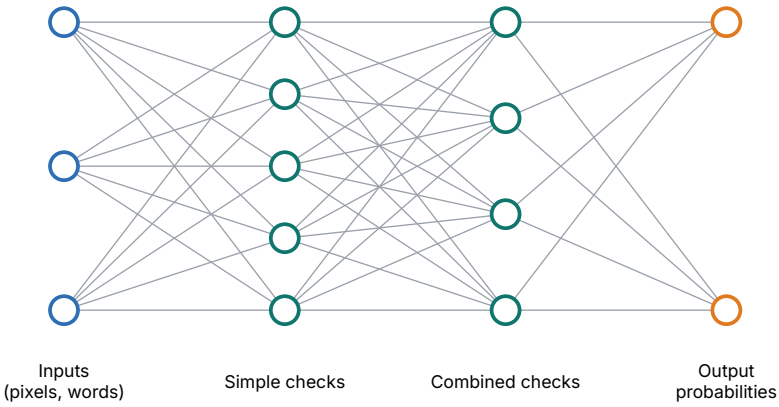


Figure 4.1. A small neural network. Each circle is a neuron: it adds up its inputs with learned weights and passes the result on. Early layers learn simple checks; later layers combine them into more complex ones.

Between layers, each neuron applies a small bend to its total before passing it on. The most common bend is very simple: if the total is negative, pass on zero; otherwise pass it on unchanged. It is called **ReLU**. Without these bends, stacking layers would add nothing, because a sum of sums is still just a sum. With them, enough neurons can learn almost any pattern.

A network with many layers is called **deep**, which is where "deep learning" gets its name. On photos, researchers found that the first layers of a trained network learn to spot edges, the middle layers learn textures and shapes, and the last layers learn whole objects (LeCun et al. 2015). For Kaveri Kart's photos, that could mean edges, then a dent or a crack, then "damaged product". Nobody wrote a rule for "dent". The network learned it from labelled photos.

● KEY IDEA

A neural network learns its own features from raw input.

That is why it can read photos, sound and free text, where nobody could write the features by hand. On clean tables of numbers, a simpler model such as gradient-boosted trees is usually just as good, and far cheaper.

How a network learns

A network starts with random weights, so its first guesses are useless. Training improves them through the same guess-check-adjust loop you saw in Chapter 3, with three new names:

1. **Loss.** For each training example, a number that says how wrong the guess was. A confident wrong answer gets a large loss; a confident right answer gets a loss near zero.
2. **Gradient descent.** Imagine standing on a hillside in thick fog, trying to reach the valley. You cannot see the valley, but you can feel which way the ground slopes under your feet, so you take a small step downhill, and repeat. The "hill" is the loss, and each step changes every weight a little in the direction that lowers it. The size of each step is the **learning rate**: too large and you overshoot the valley, too small and training takes forever.
3. **Backpropagation.** How does the network know which of its millions of weights to blame for a mistake? It passes the blame backwards, from the output layer to the first layer, giving each weight a share in proportion to how much it added to the error (Rumelhart et al. 1986). That share tells gradient descent which way to move each weight.

Training does not look at one example at a time. It takes a small group of examples, called a **batch**, works out the average loss, adjusts the weights, and moves to the next batch. One full pass through all the training examples is called an **epoch**. Small models train for a few epochs; large language models see most of their training text only once or twice.

Bigger networks become overconfident

Train a large network for a long time and something worth knowing happens. Accuracy keeps improving, or stays the same, but the model's probabilities become too extreme. Once it gets most training examples right, the only way left to lower the loss is to push its answers closer to 100%

certainty. It carries that certainty onto new examples where it has not earned it. Large modern networks are often overconfident in exactly this way (Guo et al. 2017).

This is why Chapter 2 asked you to check calibration before acting on a model's probability. A network that is right 85% of the time while saying 99% will break any threshold rule you build on top of it. The fix is usually simple: a small correction learned on validation data, which you will meet again when we test LLM judges in Part VII.

Why GPUs?

Inside a neural network, almost all the work is the same simple calculation repeated billions of times: multiply many numbers by many weights and add them up. A computer's main processor, the **CPU**, has a few very capable cores that do tasks one after another. A **GPU**, originally built to draw video-game graphics, has thousands of simple cores that all work at the same time.

Think of a CPU as a few senior chefs, and a GPU as a thousand kitchen helpers who can each only chop vegetables. For a wedding feast that needs a mountain of chopped onions, the thousand helpers win easily. Neural networks are a mountain of chopping, which is why AI runs on GPUs, and why GPU memory is one of the first limits you hit.

What a model needs in memory

A model's learned weights are called its **parameters**. A "7B model" has about 7 billion of them. Each parameter is a number, and each number takes some bytes of memory, depending on how precisely it is stored:

- **FP32** (full precision): 4 bytes per number.
- **BF16** or **FP16** (half precision): 2 bytes. This is the normal format for running large models today.
- **INT8**: 1 byte.
- **4-bit**: half a byte.

Storing numbers with fewer bytes is a bit like rounding prices to the nearest rupee instead of keeping the paise. You lose a little detail and save a lot of space. Using lower precision where it is safe is called **mixed precision**, and storing a finished model in fewer bits is called **quantisation** (Chapter 11).

Now the arithmetic is easy. To *run* a 7B model, you need at least:

- in BF16: 7 billion × 2 bytes = **14 GB**;
- in 4-bit: 7 billion × 0.5 bytes = **3.5 GB**.

On top of the weights, a running model needs extra memory for the text it is working on, which grows with the length of the conversation and the number of users at the same time. Chapter 6 shows how to estimate that part. A safe rough plan is to add 20% to 50% to the weights.

To *train* the same model is far more expensive. Training keeps, for every parameter, the weight itself, its gradient (the direction to move it), and two extra running averages used by the common optimiser, Adam (Kingma and Ba 2015). With mixed precision, this adds up to about 16 bytes per parameter (Rajbhandari et al. 2020), before counting the memory for the examples being processed:

- 7 billion × 16 bytes = **112 GB** just for the training state.

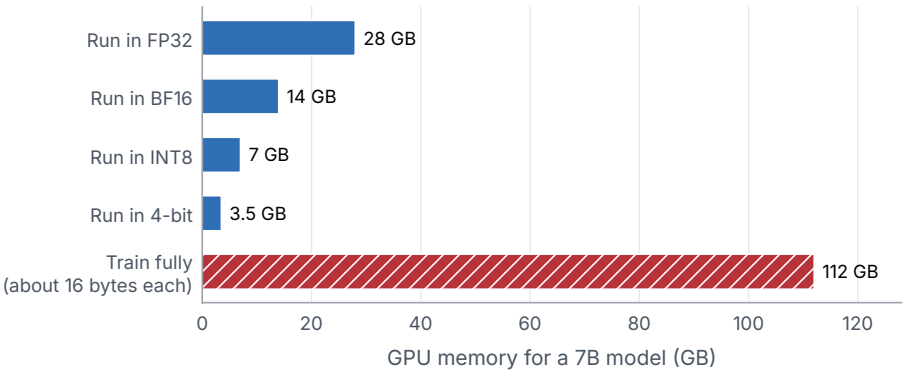


Figure 4.2. Memory needed for a 7-billion-parameter model's weights in different formats, compared with the memory needed to train it fully. Running a model is cheap compared with training it.

That is more than one large GPU holds. Figure 4.2 shows why almost nobody trains a large model fully for a business task. Instead, engineers use models someone else has trained, and when they must adapt one, they use methods that train only a small number of extra parameters. Chapter 34 covers those methods.

● KEY IDEA

Estimate memory on paper before you rent a GPU.

Running a model needs about 2 bytes per parameter in BF16, plus room for the text it is working on; fully training it needs about 16 bytes per parameter. A 7B model runs in about 14 GB but needs over 100 GB to train.

When to use deep learning yourself

As an applied AI engineer, you will almost never design a new network. The everyday choices are these:

- **Use a pre-trained model through an API** for language, vision and speech. This is the default, and most of this book is about doing it well.
- **Use a pre-trained open model on your own hardware** when data must not leave your servers, or when volume is high enough that your own GPU is cheaper than paying per token.
- **Adapt a pre-trained model** with a small amount of extra training when prompting alone cannot get the behaviour you need.
- **Use classical models** (Chapter 3) for tables of numbers and for high-volume, simple text decisions.

Training a deep network from scratch is for companies with large datasets, large budgets, and a problem no existing model handles. For Kaveri Kart, every deep learning need, from reading photos to understanding Kannada voice notes, is met by an existing model.

PROJECT STEP: KAVERI KART

1. Pick two open models you might run yourself: one around 7B parameters and one around 70B. Work out the memory for their weights in BF16 and in 4-bit.
2. Look up the memory of one common GPU you could rent. Which of the four cases fit on one GPU, after adding 30% for the working memory?
3. Write one paragraph for Kaveri Kart: should the support system use a model through an API, or run one on its own GPU? Consider the data (customer addresses and phone numbers), the volume (800 to 4,000 messages a day), and the team size. You will revisit this choice in Chapter 12 with real costs.

WHAT BREAKS IN PRODUCTION

- **The model is too big for the GPU.** The weights fit, but the working memory for long conversations and many users does not. The service crashes under load, not in testing.
- **Overconfident probabilities.** A large network's "99%" is acted on as if it were reliable. Check calibration before using any threshold.
- **Training data does not match real inputs.** A damage-photo model trained on bright studio photos fails on blurry phone photos taken in a dark corridor.
- **Deep learning where it is not needed.** A team spends weeks on a neural network for a clean table of numbers, and it does no better than gradient-boosted trees.

NUMBERS YOU MUST DEFEND

- **Memory estimate.** "The 7B model needs about 14 GB in BF16. With 30% for working memory, about 18 GB, so it fits on one 24 GB GPU."
- **Precision.** "We run in 4-bit because quality on our test set dropped by less than one point, and it lets us use a smaller GPU."
- **Why not train our own model.** "Full training would need over 100 GB of GPU memory for a 7B model, plus a large labelled dataset. Existing models already meet our quality target."

KEEP THESE

- **A neural network learns its own features from raw input.** That is why it can read photos, sound and free text, where nobody could write the features by hand. On clean tables of numbers, a simpler model such as gradient-boosted trees is usually just as good, and far cheaper.
- **Estimate memory on paper before you rent a GPU.** Running a model needs about 2 bytes per parameter in BF16, plus room for the text it is working on; fully training it needs about 16 bytes per parameter. A 7B model runs in about 14 GB but needs over 100 GB to train.

EXERCISES

1. How much memory do the weights of a 13B model need in BF16, in INT8 and in 4-bit?
2. Explain gradient descent to a friend who has never studied maths, using an example from daily life other than a hill in fog.
3. Your team wants to train a neural network to predict delivery delays from a table with 20 columns (distance, courier, weather, and so on). What would you suggest trying first, and why?

INTERVIEW QUESTIONS

1. What is backpropagation, in simple terms?
2. How much GPU memory do you need to serve a 7B model? Show your working.
3. Why do large models tend to be overconfident, and why does it matter?
4. When would you not use deep learning?

Next chapter: how computers turn words into numbers, from tokens to embeddings, and why Indian languages need special care.

CHAPTER 5

From words to tokens and vectors

A Kaveri Kart customer writes: *"mera order abhi tak nahi aaya, tracking bhi kaam nahi kar raha"* (my order has not arrived yet, and the tracking is not working either). Another writes: *"Parcel still not delivered and the tracking link is broken."* A third writes the same complaint in Hindi script.

To a person, these three messages say the same thing. To a computer, they are three completely different strings of characters, and the first one is not even written in a standard spelling. Before any model can work with language, text must become numbers, in a way that keeps the meaning. This chapter shows the two steps that every language model uses: breaking text into **tokens**, and turning tokens into **vectors** that capture meaning. Both steps affect cost, speed and quality, and both need special care for Indian languages.

IN THIS CHAPTER YOU WILL LEARN

- What a token is, how text is split into tokens, and why the same message can cost more in Hindi than in English.
- How embeddings turn text into numbers that capture meaning, and how to measure how similar two texts are.
- What embeddings miss, and why that leads to the transformer in Part II.

The problem: numbers that know nothing

Suppose you number every word in a dictionary: "bill" is 101, "invoice" is 102, "lunch" is 103. A model doing arithmetic would treat these numbers literally, and decide that "invoice" sits halfway between "bill" and "lunch". The numbers carry an order that means nothing.

Suppose instead you give every word its own position in a very long list, with a 1 in its own position and 0 everywhere else. Now no word is closer to any other. "Bill" is exactly as different from "invoice" as it is from "lunch". This is fair, but it knows nothing about meaning either.

We need two things: a sensible way to cut text into pieces, and a way to give each piece numbers so that pieces with similar meaning get similar numbers.

Tokens: the pieces a model reads

Cutting text into whole words causes trouble. There are too many words, new ones appear every day (“unboxing”, “UPI”), and every spelling mistake or Hinglish spelling becomes a new word the model has never seen. Cutting into single letters avoids this, but then even a short message becomes hundreds of pieces, which is slow and expensive.

Modern models use a middle way. They cut text into **tokens**: common words stay whole, and rarer words are split into smaller, common pieces. A model might keep “refund” as one token but split “refundable” into “refund” and “able”. A common method for learning these pieces is called **byte-pair encoding**, or **BPE** (Sennrich et al. 2016). It works like this:

1. Start with every word split into single characters.
2. Count every pair of neighbouring pieces across all the training text.
3. Join the most common pair into one new piece.
4. Repeat steps 2 and 3 until you have as many pieces as you want, often 50,000 to 200,000.

Here is BPE on a tiny example. The training text has the word “refund” 10 times, “refunded” 6 times, “refundable” 3 times, “fund” 4 times, “under” 5 times, “return” 8 times and “returned” 5 times. Counting all neighbouring pairs, the most common joins come out in this order:

1. “r” + “e” becomes “re” (it appears 32 times across all the words);
2. “u” + “n” becomes “un”;
3. “un” + “d” becomes “und”;
4. “f” + “und” becomes “fund”;
5. “re” + “fund” becomes “refund”;
6. and then “re” + “t”, and onwards towards “retur”.

After only eight joins, “refund” is one token, “refunded” is “refund” + “e” + “d”, and “return” is “retur” + “n”. With far more text and many thousands of joins, the common words of the training text become single tokens, and everything else is built from pieces.

Why tokens matter to you

Tokens affect your daily work directly. They decide three practical things:

- **Cost.** Providers charge per token (Chapter 1).
- **Limits.** Every model has a **context window**, the largest number of tokens it can read and write in one go. Long documents and long chats use it up.
- **Speed.** A model writes its answer one token at a time, so more output tokens means a longer wait.

For English, a rough rule is that 100 tokens is about 75 words. That rule does not hold for every language.

Why Indian languages can cost more

BPE learns its pieces from its training text. If that text is mostly English, the common English words become single tokens. Words in Hindi, Tamil, Kannada and other Indian languages are seen less often, so they stay split into many small pieces. The same sentence can therefore need several times more tokens in an Indian script than in English. More tokens means a higher bill, a slower reply, and less room in the context window for the same amount of meaning.

Hinglish written in Latin letters has its own problem. Words like “nahi”, “kahan” and “pichhle” are not common English words, so they are often split into odd pieces, and the same word spelled three ways becomes three different sets of tokens.

Newer tokenisers are trained on more languages, and the gap is smaller than it was a few years ago. The only reliable answer is to measure, with the tokeniser of the model you actually plan to use. For OpenAI’s models, the `tiktoken` library counts tokens on your own computer (`pip install tiktoken`):

```
import tiktoken

enc = tiktoken.get_encoding("o200k_base")    # the tokeniser of one model family
for text in ["Where is my refund?",
             "मेरा रिफंड कहाँ है?",
             "Mera refund kahan hai?"]:
    print(len(enc.encode(text)), text)
```

Other providers give a token-counting endpoint or show the count in each response, as you saw with `usage` in Chapter 1. Run the same messages

through each model on your shortlist. The differences can change which model is cheapest for your customers.

● KEY IDEA

Count tokens in the languages your customers actually use.

A tokeniser trained mostly on English can split Hindi or Kannada into many more pieces, so the same message costs more, runs slower and fills the context window sooner. Measure with each model's own tokeniser before you choose.

Embeddings: meaning as position

Now we need numbers that capture meaning. The idea came from linguistics, decades before deep learning: *"You shall know a word by the company it keeps"* (Firth 1957). Words that appear in similar places tend to mean similar things. "Refund" and "money back" both appear near "return", "credited" and "bank account". "Delivery" appears near "courier", "tracking" and "address".

An **embedding** is a list of numbers, often a few hundred to a few thousand long, given to a word or a piece of text so that texts with similar meaning get similar lists. Think of a well-run library. Books on the same subject stand on the same shelf, whatever their titles say, so you find all the cricket books in one place. An embedding gives every text a position on a huge set of shelves, and texts with similar meaning end up close together (Figure 5.1).

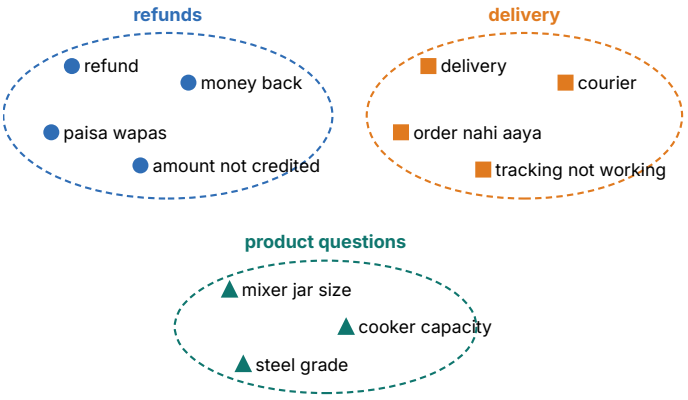


Figure 5.1. A simple sketch of embeddings, squeezed down from hundreds of numbers to two so they fit on the page. Texts with similar meaning sit close together, even when they share no words. (Illustrative positions.)

Models learn these positions from huge amounts of text. An early, famous method called word2vec trained a small network to predict each word's neighbours, and the positions it learned turned out to capture meaning surprisingly well (Mikolov et al. 2013). Modern **embedding models** do the same for whole sentences and paragraphs. You send them a text, and they return its list of numbers. You will use one in Part IV.

Measuring how close two texts are

How do we measure whether two embeddings are close? Picture each list of numbers as an arrow pointing from the centre of the library in some direction. Texts with similar meaning point in similar directions. The usual measure, **cosine similarity**, looks at the angle between two arrows:

- about **1** when they point the same way (very similar meaning);
- about **0** when they are at a right angle (unrelated);
- below 0 when they point in opposite directions, which is rare for text.

A small example with just two numbers per text makes it concrete. Suppose:

- "refund" is at (0.9, 0.2);
- "money back" is at (0.8, 0.3);
- "delivery" is at (0.1, 0.95).

Cosine similarity multiplies the matching numbers, adds them up, and divides by the lengths of the two arrows. For "refund" and "money back", that gives about **0.99**: almost the same direction. For "refund" and "delivery", it gives about **0.32**: quite different. Real embeddings have hundreds of numbers instead of two, but the calculation is the same, and libraries do it for you.

Finding similar messages

With embeddings, Kaveri Kart can do something keyword search cannot: find past messages with the same *meaning*, even when they share no words. "Paisa wapas nahi aaya" (the money has not come back) and "refund not credited to my account" can sit close together, if the embedding model has learned enough Hinglish.

This gives a simple and useful classifier, called **k-nearest neighbours**:

1. Embed all your labelled past messages once, and store the embeddings.

2. When a new message arrives, embed it.
3. Find the k past messages closest to it, for example the 10 with the highest cosine similarity.
4. Use their labels. If 8 of the 10 nearest messages were refund requests, call the new one a refund request, about 80% likely.

The same idea, finding the most similar stored texts for a question, is the heart of retrieval-augmented generation, which Part IV covers in depth.

What embeddings miss

Embeddings capture what a text is *about*. They are weak at three things you will meet often:

- **Exact numbers.** "A refund of ₹500" and "a refund of ₹50,000" are about the same topic, so their embeddings are close. If a decision depends on the amount, pass the amount separately as a number.
- **Small words that flip meaning.** "The parcel was delivered" and "the parcel was not delivered" are about the same topic and can get very similar embeddings.
- **Word order and who did what.** "The customer cancelled the order" and "the order cancelled the customer" use the same words. An embedding built from word meanings alone cannot tell which makes sense.

Understanding order and small words needs a model that reads each word in the light of the words around it. That is exactly what the **transformer** does, and it is where Part II begins.

A short history in five steps

You will hear these names in interviews and papers, so here is how they fit together:

1. **Bag of words** (before 2010): count the words, ignore their order. Simple, fast, and still a good baseline, as in Chapter 3.
2. **Word embeddings** (2013): give each word a learned position by meaning (Mikolov et al. 2013). Similar words become similar numbers.
3. **Recurrent networks** (around 2014–2017): read a sentence one word at a time, carrying a memory forward. They understood order, but forgot the start of long texts, and were slow because each word had to wait for the one before.

4. **Attention and the transformer** (2017): let every word look at every other word at once (Vaswani et al. 2017). Fast on GPUs, and good at long-range meaning.
5. **Large language models** (2018 onwards): transformers made very large and trained on enormous amounts of text. Part II explains how.

● KEY IDEA

Embeddings capture what a text is about, not the exact details.

Two messages about refunds look similar whether the amount is ₹500 or ₹50,000, and "delivered" can look like "not delivered". Pass exact facts such as amounts and dates separately, as numbers or fields.

PROJECT STEP: KAVERI KART

1. Take 20 messages from your test set and write each in three versions: English, Hindi in Devanagari, and Hinglish. Count the tokens of all 60 with the tokeniser of one model you are considering. Work out the average tokens per message for each version, and the monthly cost difference at 4,000 messages a day.
2. Using any embedding model, embed your 200 labelled messages. For five new messages, find the 10 nearest neighbours and look at their labels. Do Hinglish messages find the right neighbours?
3. Write three pairs of messages that differ only by "not" or by an amount, and check their cosine similarity. Keep these pairs; they become test cases in Part IV.

⚠ WHAT BREAKS IN PRODUCTION

- **Token bills grow with language mix.** Costs are planned on English test messages, and the real traffic is half Hindi and Hinglish.
- **The embedding model does not know the language.** An embedding model trained mostly on English places Hinglish messages almost at random, and search quietly fails for those customers.
- **Long messages are cut off.** Embedding models and context windows have length limits. A long complaint is silently cut, and the important part was at the end.
- **Numbers inside text.** Order IDs, amounts and dates are treated as ordinary words, so two different orders look nearly the same.

NUMBERS YOU MUST DEFEND

- **Tokens per message by language.** "English messages average about 40 tokens, Hinglish about 60 and Hindi about 90 on the model we chose, so our cost plan uses the real mix of languages." (Use your own measured numbers.)
- **Embedding model choice.** "We tested three embedding models on our own 200 labelled messages, including Hinglish, and chose the one whose nearest neighbours most often had the right label."
- **How many neighbours.** "We use the 10 nearest messages. With 5, results jumped around; with 20, rare types like complaints were outvoted."

KEEP THESE

- **Count tokens in the languages your customers actually use.** A tokeniser trained mostly on English can split Hindi or Kannada into many more pieces, so the same message costs more, runs slower and fills the context window sooner. Measure with each model's own tokeniser before you choose.
- **Embeddings capture what a text is about, not the exact details.** Two messages about refunds look similar whether the amount is ₹500 or ₹50,000, and "delivered" can look like "not delivered". Pass exact facts such as amounts and dates separately, as numbers or fields.

EXERCISES

1. Using the tiny BPE example in this chapter, explain why "refunded" ends up as "refund" + "e" + "d" and not as one token.
2. Work out the cosine similarity of (1, 0) and (0, 1), and of (1, 1) and (2, 2). What do the two answers mean?
3. A customer writes "I was charged ₹4,999 instead of ₹499". Why might an embedding-based search fail to find past cases about this exact problem? What would you add?

INTERVIEW QUESTIONS

1. What is a token, and why does it matter for cost?
2. How does byte-pair encoding work?
3. What is an embedding, and how do you measure similarity between two embeddings?
4. Our chatbot works well in English but poorly in Hindi. What would you investigate?

Next: Part II opens the transformer, the model that reads every word in the light of all the others, and shows how it became the large language models you use today.

Answers

Some exercises have one right answer. Others ask about your own work; for those, you will find a sample answer and what a good answer includes. Interview answers are short on purpose: they show the points an interviewer listens for, which you should then back with an example from your own project.

Chapter 1

Exercises

1. For a food-delivery support bot: *instructions and context*, a description of its job and today's offers; *knowledge*, the refund and cancellation policy and restaurant details; *tools*, looking up an order and its delivery partner's location; *tests*, a fixed set of past questions with good answers; *guardrails*, never showing a delivery partner's phone number, and a limit on refund amounts; *cost and speed*, cost per chat and the time until the first reply.
2. Input: 2,000 × ₹400 ÷ 10,00,000 = ₹0.80. Output: 200 × ₹1,600 ÷ 10,00,000 = ₹0.32. One answer costs ₹1.12. 10,000 answers a day cost ₹11,200, and a 30-day month costs ₹3,36,000.
3. A sample answer. *Act alone:* tell a customer their account balance; send a statement by email to the registered address. *Prepare for review:* reverse a disputed charge; raise a credit-card limit. *Escalate at once:* a customer reporting that their card was stolen or that money left their account without their knowledge. A good answer places actions by how costly and how reversible they are.

Interview questions

1. An ML engineer mostly trains models on the company's data and runs them in production. An applied AI engineer mostly builds products on top of existing models: choosing a model, giving it context and tools, testing it, making it safe, and shipping it. The roles overlap, but the centre of each is different.
2. First, whether the model was given the facts it needed, or is guessing. Second, whether there is a test set showing how often answers are wrong, and on which kinds of question. Third, whether the instructions tell the model to say "I don't know" when the facts are missing.

3. Estimate input and output tokens per task, including the context sent with each request (instructions, documents, history). Multiply by the provider's prices, then by expected daily volume, and add a margin for retries and long conversations. Check the estimate against real usage in the first week.

Chapter 2

Exercises

1. Of 10,000 emails, 100 are spam and the filter catches 98. Of the 9,900 normal emails, 1% are flagged by mistake: 99. So 98 + 99 = 197 emails are flagged, and 99 of them are normal. That is about **half**.
2. Within about ±8 points, so ±10 is much closer than ±2. With 50 test cases and a score near 90%, the 95% range is roughly 82% to 98%.
3. The two versions disagree on only 16 of 300 questions, and the split is 9 to 7. That is far too close to call: the difference is under one percentage point and could easily be luck. Tell your manager the versions perform the same on this test, and choose on other grounds such as cost or speed, or test on many more questions.

Interview questions

1. Not necessarily. If fraud is 1% of cases, a model that never predicts fraud is 99% accurate. Ask for precision and recall on the fraud class, and for the base rate.
2. It depends on how big a difference you need to detect. Use a paired comparison on the same questions, and bootstrap the difference. A few hundred cases usually detect a 5 to 10 point difference; a 1 to 2 point difference needs thousands.
3. Temperature controls how random the choice of each next token is. Low values make answers predictable; high values make them varied. For support, use a low value, because the same question should get the same answer.
4. A calibrated model's probabilities match reality: when it says 80%, it is right about 80% of the time. Check by grouping past cases by the probability the model gave and counting how often it was right in each group.

Chapter 3

Exercises

1. Leakage. The column "agent_approved" is filled in *after* a person reviews the request, so it contains the answer. A new request will not have it. Remove the column and retrain.
2. Approve alone when (1 − p) × ₹500 < ₹25, that is, when the chance of being wrong is below 5%. So approve alone above **95%**.
3. The model is overfitting: it has learned the training examples, not the general pattern. Try more training data, a simpler model or stronger regularisation, and check for duplicate or near-duplicate messages that make training look easy.

Interview questions

1. When the labels are fixed, there is enough labelled data, the volume is high, and you need low cost, low latency and checkable probabilities. Also for tables of numbers, where gradient-boosted trees are hard to beat.
2. Information about the answer reaching the model in training in a way it will not in real use. Examples: duplicate messages in both training and test sets, a column filled in after the outcome, or the same customer in both sets.
3. From costs: act alone when (chance of being wrong) × (cost of a wrong action) is less than the cost of asking a person. Check that the model is calibrated first, and test how much the threshold moves if the costs are halved or doubled.
4. Recall on the complaint class by itself, and which other classes complaints are confused with. Then read the missed complaints by hand.

Chapter 4

Exercises

1. 13 billion × 2 bytes = 26 GB in BF16; × 1 byte = 13 GB in INT8; × 0.5 bytes = 6.5 GB in 4-bit. Add working memory on top.
2. A sample answer: adjusting the salt in a new recipe. You taste, it is too bland, you add a little salt, taste again, and keep going in small steps until it is right. Large steps risk making it too salty.
3. Gradient-boosted trees, perhaps after a simple baseline such as logistic regression. The data is a clean table of numbers and categories, where trees usually match or beat neural networks at a fraction of the cost and effort.

Interview questions

1. A way of sharing out the blame for an error. The error at the output is passed backwards through the layers, and each weight learns how much it contributed, which tells training which way to adjust it.
2. About 2 bytes per parameter in BF16: 7 billion × 2 = 14 GB for the weights. Add working memory for the conversations being processed, roughly 20% to 50% more depending on context length and number of users. So plan for about 18 to 21 GB.
3. Training keeps lowering the loss after most answers are right by pushing probabilities towards certainty, so the model says 99% when it is right less often. Any rule that acts on its probabilities then acts too often.
4. When the data is a clean table of numbers, when you have little data, when you need to explain each decision simply, or when a pre-trained model already does the job.

Chapter 5

Exercises

1. In the training text, “refunded” appears only 6 times, while pieces like “re” and “fund” appear in many words. BPE joins the most common pairs first, so “refund” becomes one token early. The pair “refund” + “e” was never common enough to be joined within the eight steps.
2. (1, 0) and (0, 1): 0, unrelated. (1, 1) and (2, 2): 1, the same direction. Cosine similarity looks only at direction, not length.
3. Embeddings capture the topic (a wrong charge) but are weak at exact amounts, so past cases with any amounts will look similar. Extract the amounts as separate fields, and compare them directly, or combine embedding search with keyword or number search.

Interview questions

1. A small piece of text, usually a short word or part of a word, that the model reads and writes. Providers charge per token, context windows are measured in tokens, and output speed is measured in tokens per second.
2. Start with single characters, repeatedly join the most common neighbouring pair into a new piece, and stop at the desired vocabulary size. Common words become single tokens; rare words are built from pieces.

3. A list of numbers representing a text, placed so that similar meanings are close together. Measure similarity with cosine similarity: about 1 for the same direction, about 0 for unrelated.
4. Token counts in Hindi (cost and context limits), whether the embedding and language models handle Hindi and Hinglish, whether the test set includes Hindi at all, and the score on Hindi messages measured separately.

References

Alian Software. 2026. *Hiring for Production AI*. https://aliansoftware.com/blog/hiring-for-production-ai.

Firth, J. R. 1957. 'A Synopsis of Linguistic Theory, 1930–1955'. In *Studies in Linguistic Analysis*. Blackwell.

Guo, Chuan, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. 2017. 'On Calibration of Modern Neural Networks'. *Proceedings of the 34th International Conference on Machine Learning (ICML)*.

Kingma, Diederik P., and Jimmy Ba. 2015. 'Adam: A Method for Stochastic Optimization'. *International Conference on Learning Representations*.

LeCun, Yann, Yoshua Bengio, and Geoffrey Hinton. 2015. 'Deep Learning'. *Nature* 521: 436–44.

Mikolov, Tomas, Ilya Sutskever, Kai Chen, Greg S. Corrado, and Jeff Dean. 2013. 'Distributed Representations of Words and Phrases and Their Compositionality'. *Advances in Neural Information Processing Systems 26 (NeurIPS)*.

Nakrani, Alpesh. 2026. *AI Engineer Red Flags*. https://alpeshnakrani.com/blog/ai-engineer-red-flags/.

Rajbhandari, Samyam, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. 'ZeRO: Memory Optimizations Toward Training Trillion Parameter Models'. *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC20)*.

Rumelhart, David E., Geoffrey E. Hinton, and Ronald J. Williams. 1986. 'Learning Representations by Back-Propagating Errors'. *Nature* 323: 533–36.

Sennrich, Rico, Barry Haddow, and Alexandra Birch. 2016. 'Neural Machine Translation of Rare Words with Subword Units'. *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, 1715–25.

Vaswani, Ashish, Noam Shazeer, Niki Parmar, et al. 2017. 'Attention Is All You Need'. *Advances in Neural Information Processing Systems 30 (NeurIPS)*.

Wang, Shawn. 2023. *The Rise of the AI Engineer*. Latent Space. https://www.latent.space/p/ai-engineer.

About the author

Sridhar Mukkandi is an applied AI engineer who builds AI agents and the decision systems behind them. He writes for engineers who want AI systems they can trust in production.

sridhar.authorhub@gmail.com